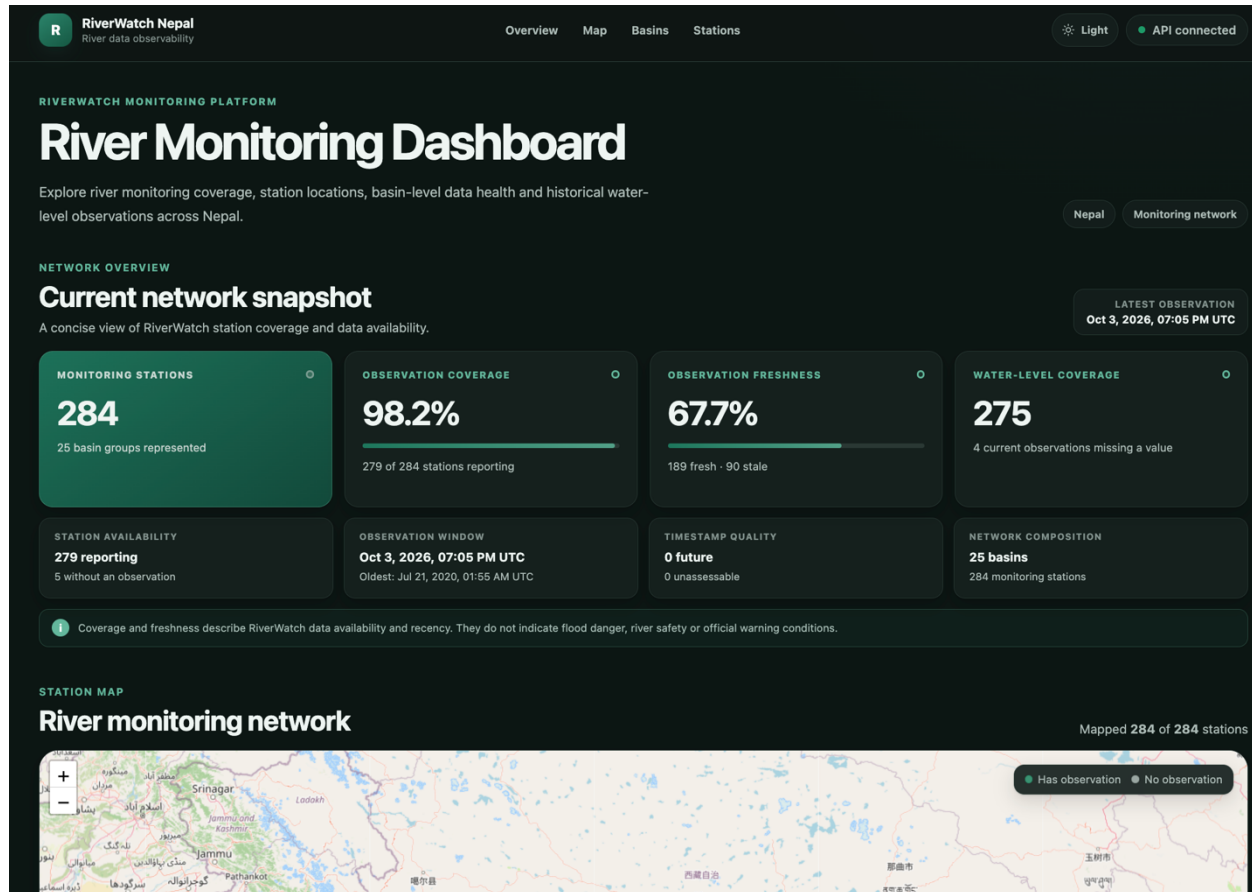


RiverWatch Nepal

Project Overview

River Monitoring and Analytics Platform



Nishanta Nepal

MSc Computer Science

Technical University Dresden

nishanta.nepal@mailbox.tu-dresden.de

1. Executive summary

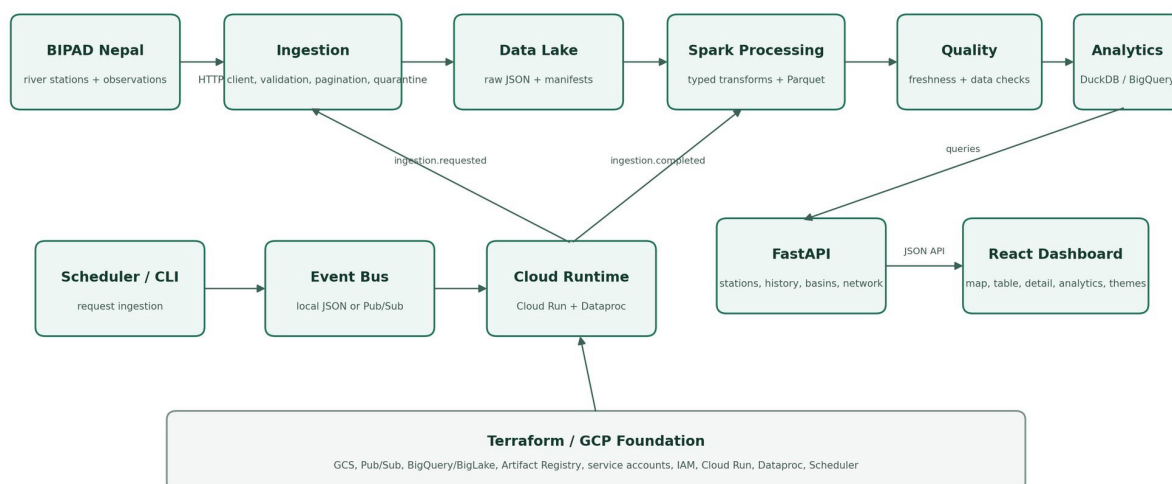
RiverWatch Nepal is a cloud-ready river-monitoring data platform built around Nepal BIPAD river-station data. The project grew from a small source-ingestion experiment into a layered system that preserves source evidence, processes and validates data, exposes typed analytics through an API, renders an interactive dashboard, and defines an event driven GCP deployment architecture as Terraform.

Current system status

The repository is synchronized at revision 42a8776. The application is validated locally and deployment-ready, but billable GCP resources have intentionally not been applied. The documented cloud path is therefore an infrastructure/runtime contract rather than a currently hosted public service.

RiverWatch Nepal - Current System Architecture

Local-first data engineering with cloud-ready event-driven runtime contracts



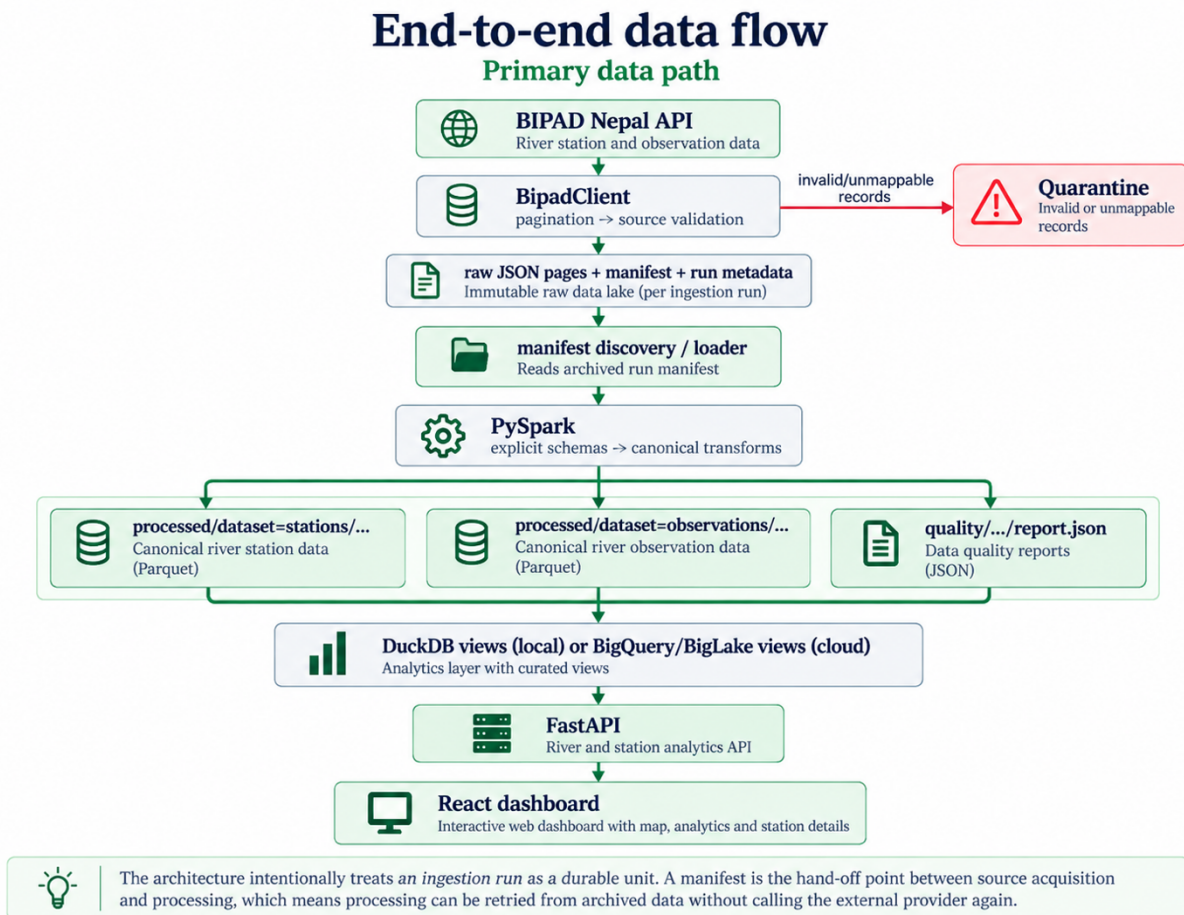
2. What RiverWatch provides

- Reliable BIPAD station/observation ingestion with retries pagination, explicit source schemas, validation, mapping, quarantine and metrics.
- An immutable raw lake that keeps original source responses and runs manifests before any transformation occurs.
- Spark-based canonical station and observation datasets written as Parquet using explicit schemas and deterministic paths.
- Separate data-quality output that evaluates station fields, observation completeness, timestamp validity and freshness without rewriting source evidence.
- Local DuckDB analytics and cloud BigQuery analytics behind the same typed service boundary.
- A FastAPI HTTP layer for stations, history, basins, network summaries and health.
- A React/TypeScript dashboard with network KPIs, a station map, station history, basin analytics, station table, error boundaries and light/dark themes.
- An event-driven execution path with ingestion/processing events, idempotency, retries, dead-letter handling and durable completion receipts.
- Terraform and container definitions for GCS, Pub/Sub, Cloud Run, Dataproc, BigQuery/BigLake, Artifact Registry, Cloud Scheduler and IAM.

3. Core design principles

Principle	How it appears in RiverWatch
Preserve evidence first	Raw BIPAD responses are archived before transformation so downstream bugs can be diagnosed against original source material.
Explicit contracts	Pydantic models, Spark schemas, event envelopes and API schemas make boundaries intentional and testable.
Local-first, cloud-ready	The same architecture can run with filesystem + DuckDB locally and GCS + BigQuery/Pub/Sub/Dataproc in the cloud.
Quality is separate from source	Bad or stale data is reported/quarantined; it is not silently “fixed” in a way that erases provenance.
Idempotency and recovery	Duplicate event delivery and retried processing should reuse durable state rather than duplicate work or emit inconsistent completions.
Least privilege and cost control	Terraform avoids broad IAM patterns, keeps event/API/schedule features disabled by default, and supports controlled activation.

4. End-to-end data flow



5. Development timeline

Phase	Focus	Key commits
1	Foundation and BIPAD data contracts	b840ae3, 9186eab
2	Reliable BIPAD ingestion	46a5e83
3	Immutable local data lake	5d2d376
4	Spark processing and data quality	baeb78e
5	Analytics layer	d85296e
6	HTTP serialization and FastAPI	edb2a5c, c372dd3
7	React monitoring dashboard	c78fc30, e99564e, c1dd4ce, 3f521f8, 42a8776
8	Event-driven local pipeline	900187c, 7072d9d, fe22d0d, a77e8a5
9	Terraform and GCP infrastructure contracts	a3bf491, d33cddb, bfebf08, f3ddf45, d7163dc, a3417c6
10	Cloud runtime configuration and durable ingestion runs	e9b6e71, 51c4ba4, 19e3feb
11	Managed cloud processing, durable completion, and BigQuery API	149a3c6, 195beb0
12	Deployment readiness and final system polish	42a8776

6. Foundation and BIPAD data contracts

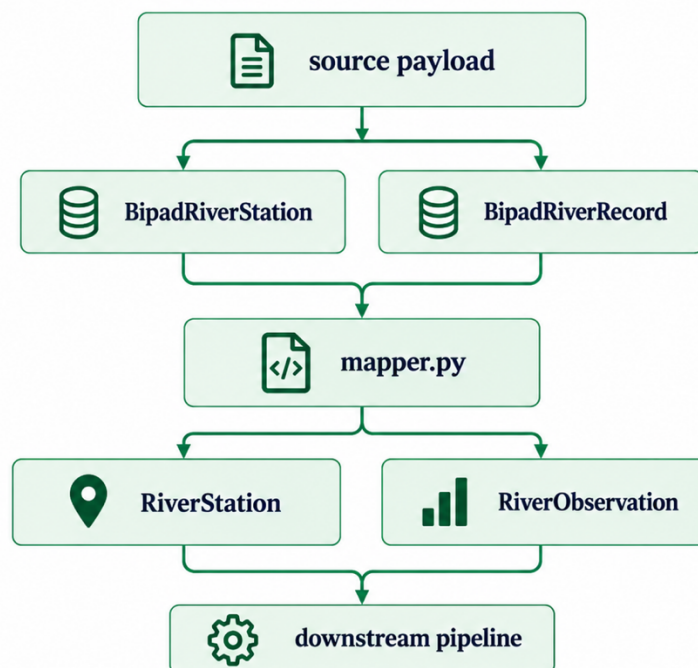
Established the Python project, configuration, source/provider models, canonical river/station models, and typed BIPAD source schemas so later layers could depend on stable contracts.

Commit	Role in this phase
b840ae3	Initialized RiverWatch project
9186eab	Modeled BIPAD river data contracts

Representative files `src/riverwatch/config.py`; `src/riverwatch/models/`;
`src/riverwatch/ingestion/models.py`; `src/riverwatch/ingestion/bipad/schemas.py`;
`src/riverwatch/ingestion/bipad/endpoints.py`

The first engineering problem was not “how to draw a dashboard”; it was how to represent source records without tying every later layer directly to BIPAD fields. Canonical station, river-observation, provider and source models created that separation. BIPAD Pydantic schemas represent what the provider sends; mapper functions translate them into the internal domain.

Canonical mapping flow Source payloads to canonical domain models



7. Reliable BIPAD ingestion

Built the HTTP client, retries, pagination, validation, mapping, logging/metrics, quarantine behavior, and ingestion services needed to ingest station snapshots and observations safely.

Commit	Role in this phase
46a5e83	Completed reliable BIPAD ingestion pipeline

Representative files

`src/riverwatch/ingestion/bipad/client.py; mapper.py; pagination.py; validation.py; service.py; quarantine_writer.py`

The ingestion client separates a single HTTP attempt from retries orchestration. Timeouts/request failures and selected HTTP status codes are retryable; decode/validation problems are handled as data or response errors instead of endlessly retrying a bad payload. The run uses one ingestion timestamp, so a batch remains internally coherent.

8. Immutable local data lake

Introduced storage abstractions and deterministic lake paths so every ingestion run preserves raw source pages, manifests, metadata, checksums, and quarantined records without silently overwriting prior evidence.

Commit	Role in this phase
5d2d376	Added immutable local data lake

Representative files

`src/riverwatch/storage/object_store.py; local.py; paths.py; raw.py; manifest.py; serialization.py`

Storage is addressed through an object-store protocol instead of calling the local filesystem directly from ingestion. That decision later allowed GCS to replace the local store without changing the domain workflow. Paths encode provider, endpoint, UTC time and run identity; manifests describe exactly which raw objects belong to the run.

9. Spark processing and data quality

Added manifest-driven raw-data discovery, explicit Spark schemas, transformations, Parquet output, recovery/idempotent processing behavior, and station/observation/freshness quality reports.

Commit	Role in this phase
baeb78e	Added Spark raw-to-Parquet processing pipeline

Representative files

`src/riverwatch/processing/; src/riverwatch/processing/spark/; src/riverwatch/quality/`

Spark does not infer the source contract blindly. Explicit StructType schemas make source drift visible and transformations predictable. Processing writes separate station and observation Parquet datasets and a quality report. Recovery logic checks durable outputs so a retry can avoid rewriting a completed result.

10. Analytics layer

Built DuckDB-backed analytical views and typed services for the current station snapshot, station history, basin summaries, and whole-network health while preserving stations with missing observations.

Commit	Role in this phase
d85296e	Added analytics layer: DuckDB, analytical views, current river snapshot, historic station time-series, basin and aggregate analytics, analytics query service

Representative files `src/riverwatch/analytics/catalog.py`; `views.py`; `service.py`; `model.py`; `protocol.py`

Analytics was designed around the questions the dashboard/API need rather than exposing raw Parquet files. The current snapshot keeps all 284 stations, including stations with no current observation. Historical views deduplicate repeated ingestion runs by station/timestamp, and basin/network views aggregate availability and freshness.

Validated snapshot example

The typed analytics service reported 284 stations across 25 basin groups, 279 stations with an observation, 5 without an observation, 275 observations with a water-level value, and a coverage ratio of about 98.2%. Snapshot values are data-dependent and change with later ingestions.

11. HTTP serialization and FastAPI

Separated internal domain models from external HTTP schemas, then exposed health, station, observation-history, basin, and network-summary endpoints through a dependency-injected FastAPI application.

Commit	Role in this phase
edb2a5c	Created a HTTP serialization boundary
c372dd3	Added FastAPI API layer: API response schema, river/station endpoints, historic time-series endpoint, basin/network endpoints

Representative files `src/riverwatch/api/app.py`; `dependencies.py`; `routes/`; `schemas/`

The HTTP serialization boundary prevents internal dataclasses/Pydantic models or database rows from leaking directly into the public API. Route handlers depend on an analytics protocol/factory, which is why the same API can be backed by local DuckDB during development and BigQuery in the cloud.

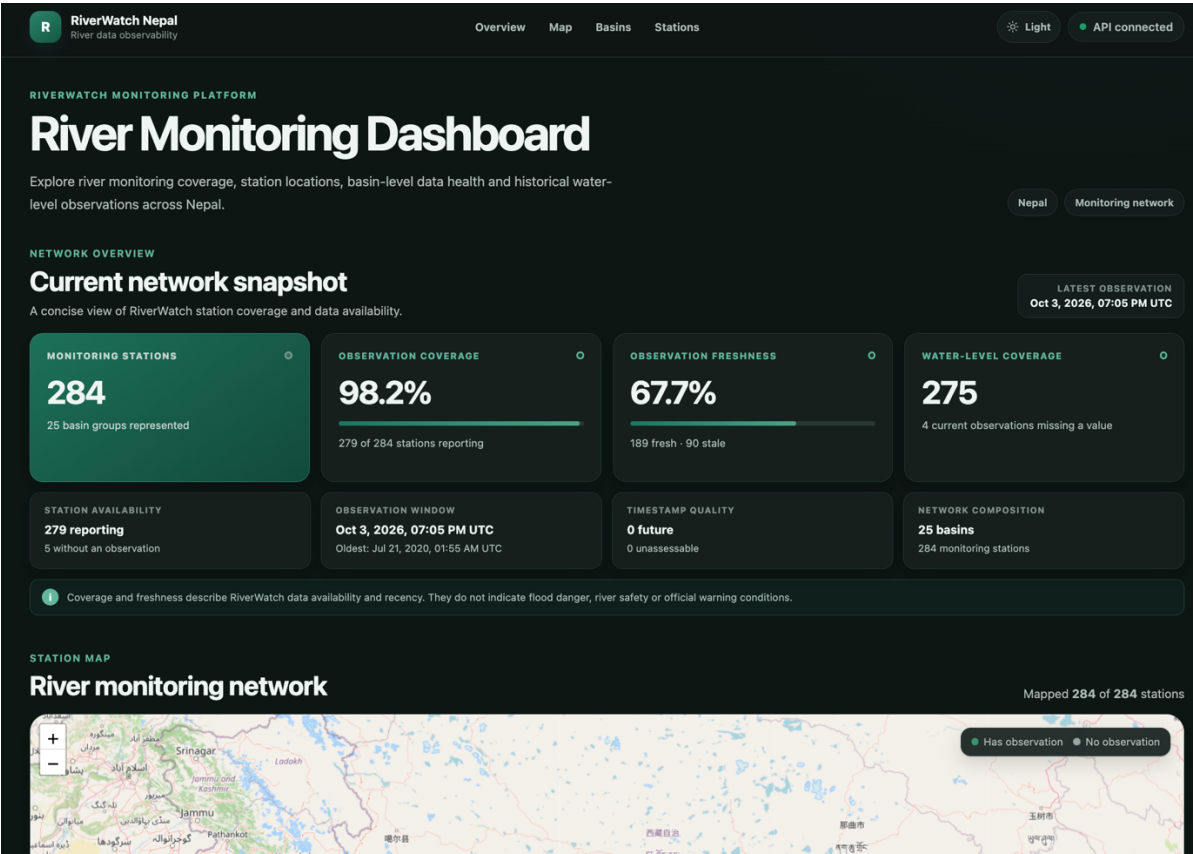
12. React monitoring dashboard

Created the TypeScript/Vite client, API adapters, network overview, searchable station table, interactive Leaflet map, station detail/history chart, basin analytics, error boundaries, tests, and later the final visual redesign with light/dark themes.

Commit	Role in this phase
c78fc30	Added frontend foundation TypeScript and API connectivity and made a Network overview dashboard with Station table + search/filtering
e99564e	Added a Interactive station map
c1dd4ce	Added Station detail + historical chart
3f521f8	Basin analytical dashboard created and frontend tested
42a8776	(HEAD -> main, origin/main) Dashboard designed and added dark mode

Representative files frontend/src/App.tsx; frontend/src/api/; frontend/src/components/; frontend/src/dashboard-refresh.css

The frontend evolved incrementally instead of being built as one large component. First came API connectivity, the network overview and station table; then the interactive map; then station detail/history; then basin analytics and tests; finally, the interface was redesigned around a persistent header, clearer hierarchy, responsive layout and a stored light/dark theme.



RiverWatch Nepal
River data observability

OverviewMapBasinsStations

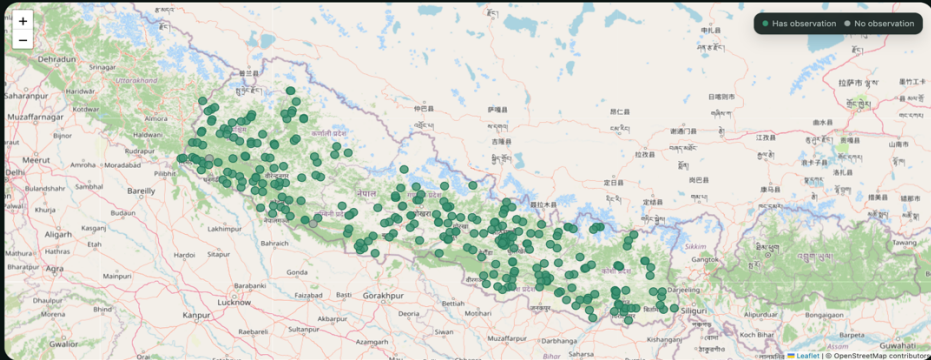
LightAPI connected

River monitoring network

Mapped 284 of 284 stations

Has observation

No observation



Marker status represents data availability only. It does not indicate flood severity or river safety.

STATION DETAIL

Explore a station

Select a station from the map or station table to view its current observation and historical water-level data.

BASIN ANALYTICS

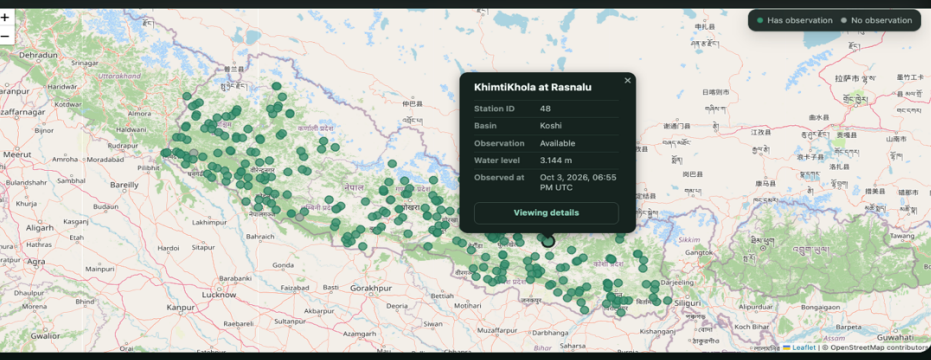
Network by basin

25 basin groups

RiverWatch Nepal
River data observability

OverviewMapBasinsStations

LightAPI connected



Has observation

No observation

Close

KhimtiKhola at Rasnalu

Station ID48

BasinKoshi

ObservationAvailable

Water level3.144 m

Observed atOct 3, 2026, 06:55 PM UTC

Viewing details

Marker status represents data availability only. It does not indicate flood severity or river safety.

STATION DETAIL

KhimtiKhola at Rasnalu

Station 48

STATION INFORMATION

Basin	Koshi
Latitude	27.57600
Longitude	86.20000
Elevation	Not available

CURRENT OBSERVATION

Status	Available
Water level	3.144 m
Observed at	Oct 3, 2026, 06:55 PM UTC

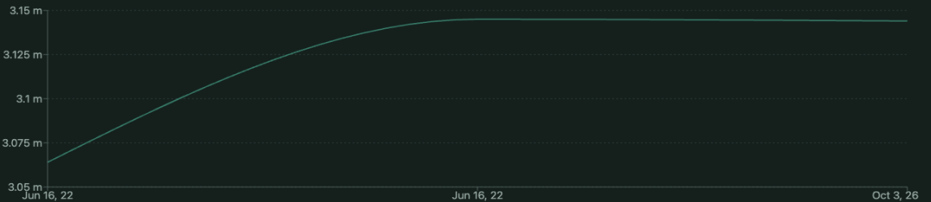
HISTORICAL DATA

Observations	3
Water-level values	3
First	Jun 16, 2022, 05:15 AM UTC
Latest	Oct 3, 2026, 06:55 PM UTC

HISTORICAL OBSERVATIONS

Water level over time

3 water-level values



The chart displays observations stored by RiverWatch. Missing water-level values create gaps. Values are analytical observations and are not flood-safety classifications.

BASIN ANALYTICS

Network by basin

13. Event-driven local pipeline

Introduced domain event envelopes, publisher abstractions, scheduled ingestion requests, workers, idempotency records, retry/dead-letter behavior, and local smoke/integration tests so ingestion and processing could be decoupled.

Commit	Role in this phase
900187c	Created Event contracts + publisher abstraction
7072d9d	Scheduled ingestion request workflow and ingestion worker + ingestion completed
fe22d0d	Created Idempotency + duplicate-delivery protection
a77e8a5	Completed event-driven pipeline: retires + dead-letter handling, local worker/scheduler cli, smoke and regression test

Representative files

```
src/riverwatch/events/; src/riverwatch/ingestion/bipad/event_runner.py; scripts/request_ingestion.py;
scripts/run_ingestion_worker.py; scripts/run_processing_worker.py
```

The event layer decouples “request work” from “perform work.” Local development stores event JSON files; cloud runtime uses Pub/Sub. Event IDs, correlation metadata, idempotency records and retry state are durable so duplicate delivery is expected rather than treated as impossible.

14. Terraform and GCP infrastructure contracts

Defined project services, least-privilege service accounts, Pub/Sub, dead-letter policies, GCS lake resources, BigQuery/BigLake, Cloud Scheduler, Artifact Registry, Cloud Run and Dataproc infrastructure, plus validation and security/cost guardrails.

Commit	Role in this phase
a3bf491	Terraform foundation and created GCP APIs and dedicated runtime identities and least-privilege IAM foundation
d33cdbb	Added Pub/Sub topics, subscriptions, retries, and dead-letter topics
bfebf08	BigQuery dataset + BigLake connection + IAM + External table/schema contracts
f3ddf45	Cloud scheduler + artifact registry created
d7163dc	Networking + runtime identities + Cloud Run/Dataproc infrastructure contracts
a3417c6	Terraform validation, architecture docs, security/cost hardening

Representative files `infra/terraform/modules/; infra/terraform/stacks/dev/;`
`infra/terraform/schemas/bigquery/`

Terraform was intentionally built before any production deployment. Modules capture reusable resource contracts and the dev stack composes them. Security checks reject privileged IAM patterns, public principals, Dataproc service-agent role misuse and infrastructure patterns that would make Spark batches Terraform-owned long-lived resources.

Cost-control default

The example/dev feature flags for scheduled ingestion, Cloud Run event services and the API are false by default. This lets the infrastructure validate without forcing a billable deployment.

15. Cloud runtime configuration and durable ingestion runs

Added cloud environment contracts, shared GCP runtime foundations, GCS-backed object/run repositories, and recovery semantics so cloud workers can resume completed or partial ingestion attempts without repeating unnecessary source fetches.

Commit	Role in this phase
e9b6e71	Created cloud runtime configuration contracts
51c4ba4	Add shared GCP runtime foundations
19e3feb	Added durable ingestion run repositories

Representative files

```
src/riverwatch/cloud/config.py; src/riverwatch/storage/gcs.py; src/riverwatch/ingestion/bipad/gcs_run_repository.py;
run_repository.py
```

Cloud runtime code keeps configuration explicit and pushes storage/run-state behind interfaces. Durable run repositories distinguish a completed run from a partial attempt; a repeated request can reuse the completed run or move to the next attempt without blindly refetching source data.

16. Managed cloud processing, durable completion, and BigQuery API

Implemented Cloud Run ingestion/processing services, Dataproc batch submission, managed Spark execution, durable processing-completion receipts, a GCS-finalize completion relay, container contracts, and a BigQuery-backed implementation of the analytics API.

Commit	Role in this phase
149a3c6	Added managed cloud processing and completion relay
195beb0	Added BigQuery-backed cloud API

Representative files

```
src/riverwatch/cloud/; containers/; src/riverwatch/analytics/bigquery.py;
scripts/check_processing_completion_relay_contract.sh
```

Managed processing became a multi-service flow. A processing gateway receives the ingestion-completed event, validates it and submits a Dataproc batch. The Spark job writes processed and quality outputs and persists the canonical processing-completion EventEnvelope as a durable receipt. A GCS OBJECT_FINALIZE notification invokes a dedicated completion relay that reads that receipt and publishes processing.completed.

17. Deployment readiness and final system polish

Validated linux/amd64 container builds, Terraform readiness, backend regression checks, frontend lint/test/build gates, and finalized the dashboard design. Actual billable GCP deployment remains intentionally paused.

Commit	Role in this phase
42a8776	(HEAD -> main, origin/main) Dashboard designed and added dark mode

Representative files

containers/; scripts/check_terraform.sh; frontend/src/App.tsx; frontend/src/dashboard-refresh.css

Deployment readiness focused on proving that the artifacts could be deployed later without actually enabling paid GCP services. linux/amd64 images were built locally for the API, ingestion service, processing gateway, completion relay and Spark runtime. Terraform format/validation and custom policy checks passed. The dashboard redesign and its tests/build completed the current system snapshot.

18. Current repository architecture

Area	Purpose
src/riverwatch/	Backend domain, ingestion, storage, processing, quality, analytics, API, events and cloud adapters.
frontend/	React + TypeScript monitoring dashboard.
infra/terraform/	Reusable GCP modules and the composed dev stack.
containers/	Container contracts for API, ingestion, processing gateway, completion relay and Spark.
scripts/	Inspection, local execution, backfill, worker, Terraform and contract validation utilities.
tests/	Unit and integration coverage across backend boundaries.
docs/	Focused architecture decisions, analytics/API/quality/event/cloud/Terraform documentation.

19. Testing and verification

Reliability was verified continuously rather than only at the end. The final backend regression run reached 484 passing tests. The Python codebase also passed Ruff and strict mypy checks over 114 source files. Terraform format/validation/policy checks and the processing-completion-relay contract check passed. The current frontend gate contains six passing Vitest tests, ESLint, TypeScript compilation and a Vite production build.

Verification	Current evidence
Backend tests	484 passed in the full regression run.
Static analysis	Ruff clean; strict mypy clean across 114 source files.
Terraform	fmt/validate and custom IAM/public-principal/Dataproc policy checks passed.
Completion relay	Container/runtime/Terraform ownership contract script passed.
Frontend	6 Vitest tests passed; ESLint passed; TypeScript + Vite production build passed.
Frontend coverage	NetworkOverview has high direct coverage; total frontend coverage remains lower because map/detail/basin components do not yet

20. Cloud deployment model

The dev Terraform stack defines the resources needed for a cloud deployment, but deployment is deliberately featuregated and currently unapplied. The intended cloud data path is GCS for the lake, Pub/Sub for events, Cloud Run for HTTP/event services, Dataproc for managed Spark, BigQuery/BigLake for analytics, Artifact Registry for images, and Cloud Scheduler for periodic ingestion requests.

Not a live hosted service

Cloud deployment was paused because billing was intentionally not enabled. Documentation therefore distinguishes “deployment-ready” from “deployed.” No README or PDF should imply RiverWatch is currently running continuously in GCP.

21. Current limitations and boundaries

- RiverWatch is a monitoring/data-observability system; station marker state and freshness are not flood-severity predictions.
- The cloud architecture is validated as code and through local container/contracts but has not been applied as a live GCP environment.
- Frontend automated coverage is intentionally smaller than backend coverage, especially for map/chart-heavy components.
- Source quality depends on BIPAD availability and content; RiverWatch preserves and reports quality problems rather than inventing missing measurements.
- The current project does not implement the separate future mobile/geolocation/alert concept.

22. Commit history

Revision	Commit message
b840ae3	Initialized RiverWatch project
9186eab	Modeled BIPAD river data contracts
46a5e83	Completed reliable BIPAD ingestion pipeline
5d2d376	Added immutable local data lake
baeb78e	Added Spark raw-to-Parquet processing pipeline
b93e747	Add README for RiverWatch Nepal project
d85296e	Added analytics layer:DuckDB, analytical views, current river snapshot, historic station time-series, basin and aggregate analytics, analytics query service
edb2a5c	Created a HTTP serialization boundary
c372dd3	Added FastAPI API layer: API response schema, river/station endpoints, historic time-series endpoint, basin/network endpoints
c78fc30	Added frontend foundation TypeScript and API connectivity and made a Network overview dashboard with Station table + search/filtering
e99564e	Added a Interactive station map
c1dd4ce	Added Station detail + historical chart
3f521f8	Basin analytical dashboard created and frontend tested

900187c	Created Event contracts + publisher abstraction
7072d9d	Scheduled ingestion request workflow and ingestion worker + ingestion completed
fe22d0d	Created Idempotency + duplicate-delivery protection
a77e8a5	Completed event-driven pipeline retires + dead-letter handling, local worker/scheduler cli, smoke and regression test
a3bf491	Terraform foundation and created GCP APIs and dedicated runtime identities and least-privilege IAM foundation
d33cdbb	Added Pub/Subtopics, subscriptions, retries, and dead-letter topics
bfebf08	BigQuery dataset + BigLake connection + IAM + External table/schema contracts
f3ddf45	Cloud scheduler + artifact registry created
d7163dc	Networking + runtime identities + Cloud Run/Dataprocc infrastructure contracts
a3417c6	Terraform validation, architecture docs, security/cost hardening
e9b6e71	Created cloud runtime configuration contracts
51c4ba4	Add shared GCP runtime foundations
19e3feb	Added durable ingestion run repositories
149a3c6	Added managed cloud processing and completion relay
195beb0	Added BigQuery-backed cloud API
42a8776	(HEAD -> main, origin/main) Dashboard designed and added dark mode

23. Conclusion

RiverWatch now demonstrates an end-to-end data-platform lifecycle: external API ingestion, immutable raw storage, typed distributed processing, quality assessment, analytics modeling, HTTP API design, interactive visualization, event driven reliability, infrastructure as code, containerization and deployment readiness. The project remains intentionally local-first so every layer can be understood and tested without requiring continuous cloud spend.